

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Modul 1: PENGANTAR MIKROKONTROLER

Nicholas Benaya - 5024241050

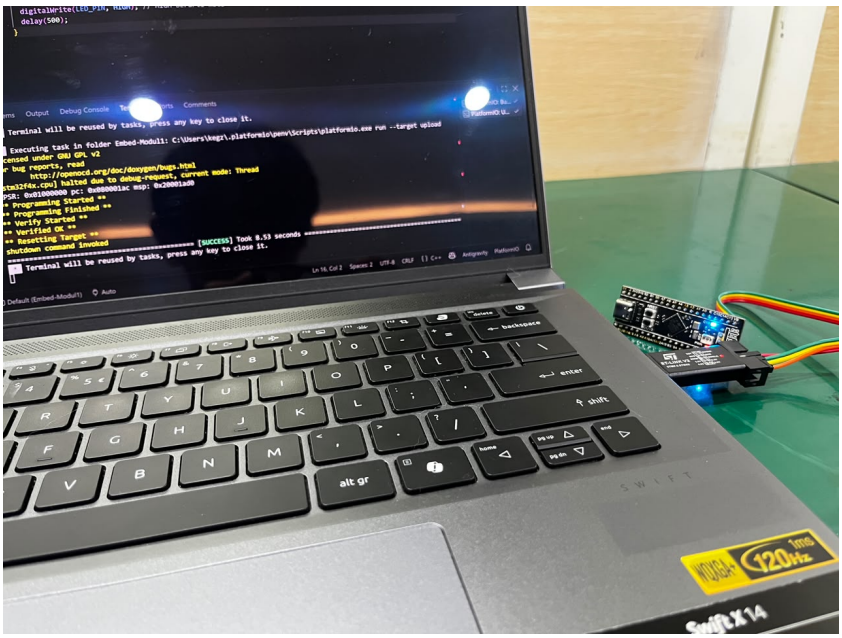
2026

1 Langkah-Langkah Percobaan

Berikut adalah tahapan percobaan pada Modul 1, meliputi pengujian hardware di laboratorium dan pengujian lanjutan di simulator Wokwi.

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink Arduino)

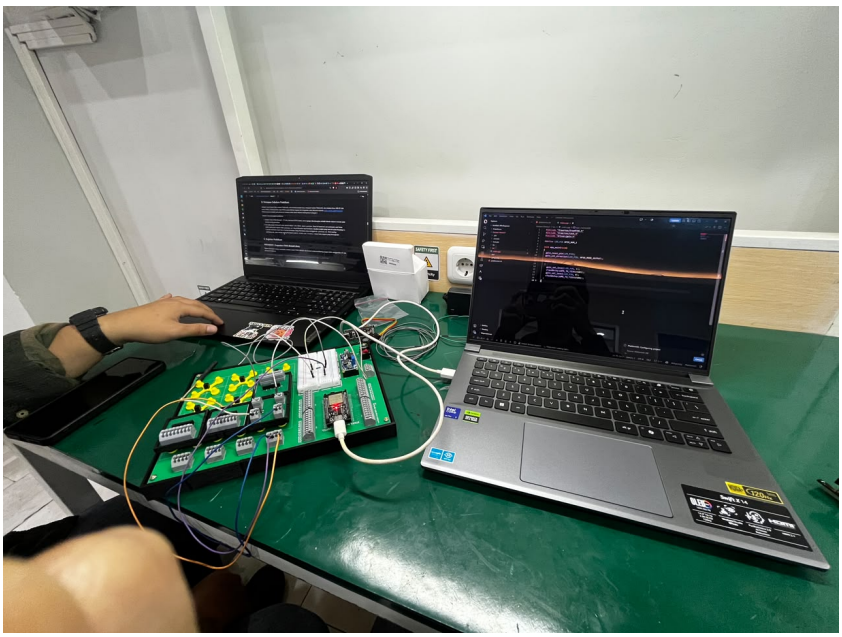
1. Menyiapkan board STM32 Blackpill (STM32F401CCU6) dan modul programmer ST-Link V2.
2. Menghubungkan pin ST-Link ke Blackpill: SWDIO ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke pin 3V3.
3. Mengatur berkas `platformio.ini` dengan konfigurasi board `blackpill_f401cc`, framework `arduino`, dan protokol upload `stlink`.
4. Menulis kode Blink pada `src/main.cpp` untuk mengedipkan LED onboard di pin PC13 dengan delay 500 ms.
5. Mengunggah kode via ST-Link dan mendokumentasikan kedipan LED onboard seperti pada Gambar 1.



Gambar 1: [HARDWARE] Hasil Uji Coba Blink pada STM32 Blackpill via ST-Link

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

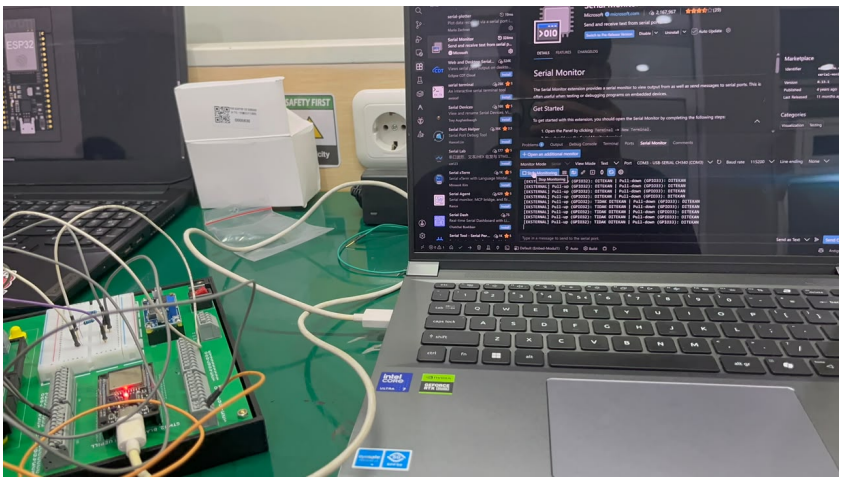
1. Menghubungkan board ESP32 ke komputer menggunakan kabel data micro USB.
2. Mengubah berkas `platformio.ini` menjadi board `esp32dev` dan framework `espidf`.
3. Menghapus file `src/main.cpp` dan membuat file C murni `src/main.c`.
4. Menuliskan logika Blink menggunakan fungsi bawaan ESP-IDF (`gpio_set_direction` dan `gpio_set_level`) serta fungsi delay `vTaskDelay()` di dalam fungsi utama `app_main()`.
5. Mengunggah program ke ESP32 dan mendokumentasikan LED onboard GPIO 2 yang berkedip (Gambar 2).



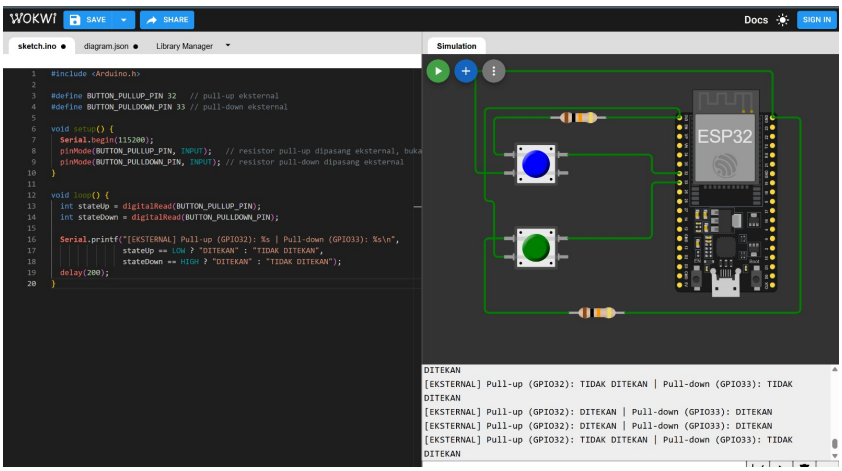
Gambar 2: [HARDWARE] Hasil Uji Coba Blink pada ESP32 Menggunakan Framework ESP-IDF

1.3 Percobaan 3: Pull-up dan Pull-down Eksternal dan Internal

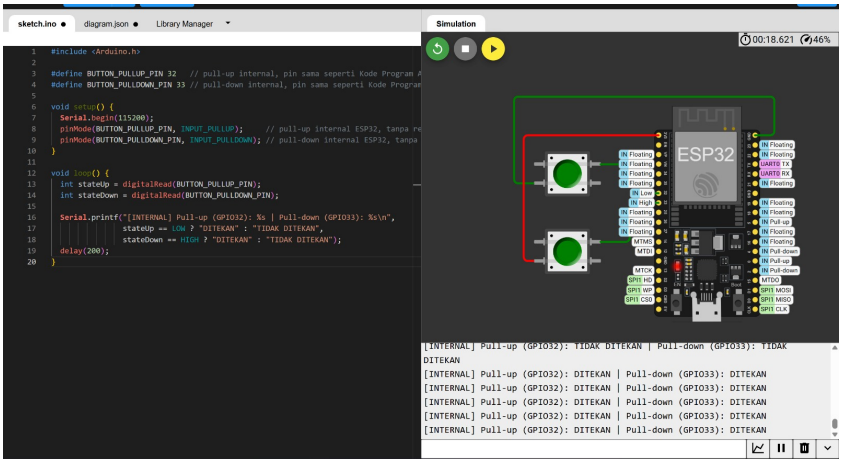
1. Mengembalikan pengaturan `platformio.ini` ke framework `arduino` dan menggunakan berkas `src/main.cpp`.
2. **Pengujian Eksternal (Praktikum Lab):** Memasang dua tombol pada breadboard dengan resistor 10kΩ eksternal (Pushbutton 1 pull-up pada GPIO 32 dan Pushbutton 2 pull-down pada GPIO 33). Saat praktikum di lab, terjadi kesalahan orientasi kaki tombol pull-down sehingga output terbaca ditekan terus-menerus (Gambar 3).
3. **Pengujian Eksternal (Perbaikan Simulasi):** Menguji ulang rangkaian pull-up dan pull-down eksternal di simulator Wokwi dengan posisi kaki tombol yang benar (Gambar 4).
4. **Pengujian Internal (Simulasi):** Menguji fitur pull-up dan pull-down internal ESP32 tanpa resistor eksternal menggunakan mode pin `INPUT_PULLUP` pada GPIO 32 dan `INPUT_PULLDOWN` pada GPIO 33 via Wokwi (Gambar 5).



Gambar 3: [HARDWARE] Hasil Praktikum Percobaan 3 Eksternal di Lab (Salah Orientasi Tombol Pull-down)



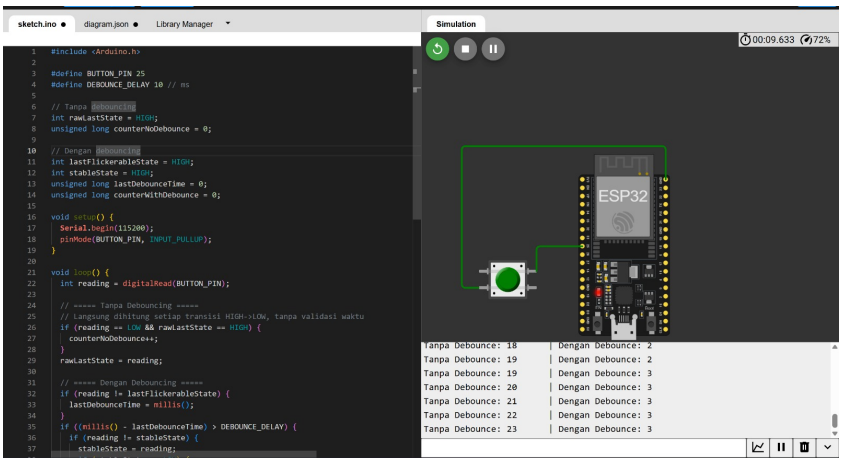
Gambar 4: [SIMULASI WOKWI] Hasil Perbaikan Rangkaian Pull-up dan Pull-down Eksternal



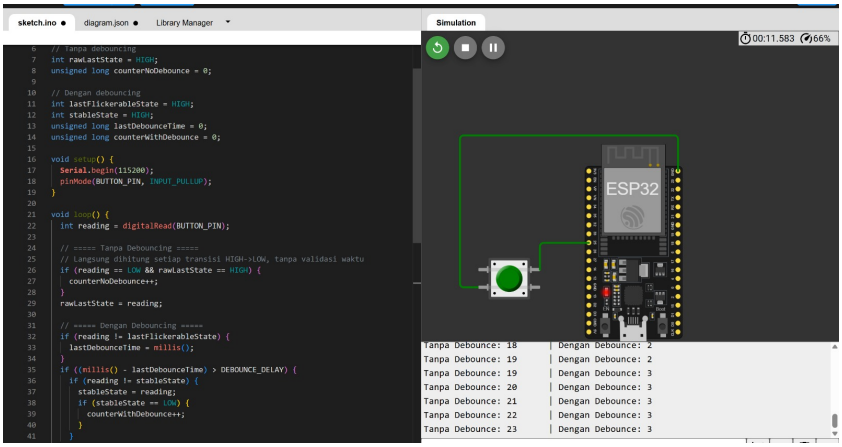
Gambar 5: [SIMULASI WOKWI] Hasil Pengujian Pull-up dan Pull-down Internal ESP32

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO

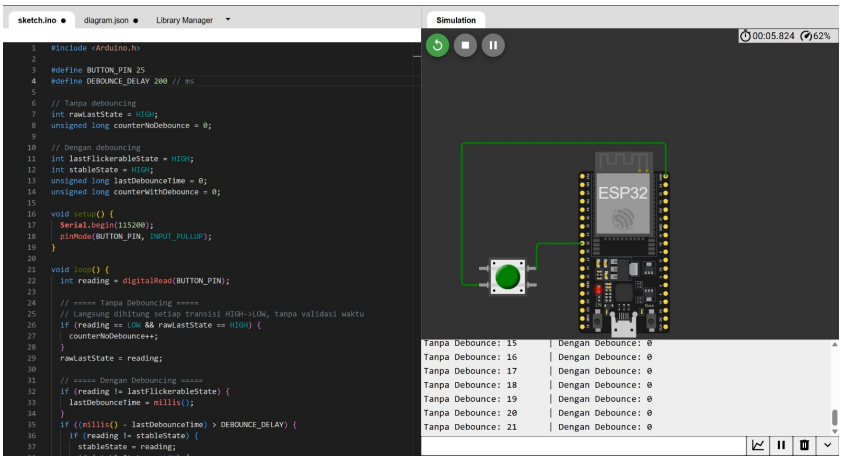
1. Menyiapkan rangkaian satu pushbutton pada GPIO 25 menggunakan mode internal `INPUT_PULLUP` di Wokwi.
2. Menjalankan kode pembanding dua variabel pencacah: `counterNoDebounce` (tanpa filter waktu) dan `counterWithDebounce` (dengan algoritma stabilisasi waktu berbasis `millis()`).
3. Menguji respon pencacah tombol pada tiga variasi nilai delay debounce:
    - Pengujian debounce delay 10 ms (Gambar 6).
    - Pengujian debounce delay 50 ms (Gambar 7).
    - Pengujian debounce delay 200 ms (Gambar 8).



Gambar 6: [SIMULASI WOKWI] Pengujian Debouncing Tombol dengan Delay 10 ms



Gambar 7: [SIMULASI WOKWI] Pengujian Debouncing Tombol dengan Delay 50 ms

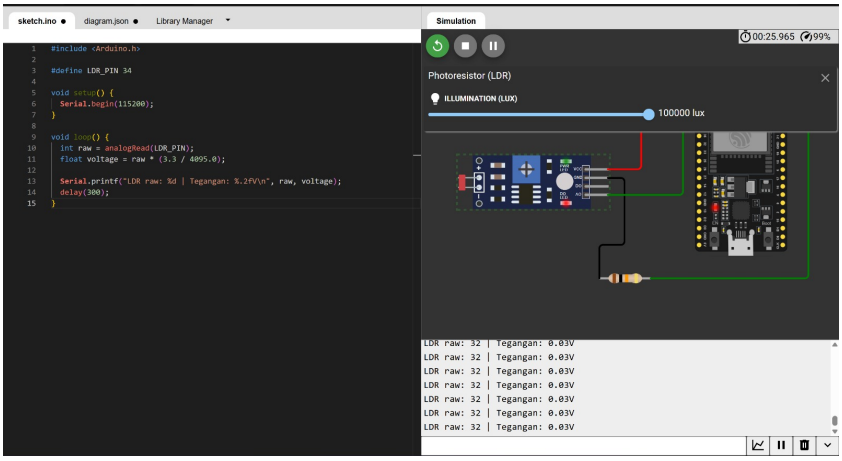


Gambar 8: [SIMULASI WOKWI] Pengujian Debouncing Tombol dengan Delay 200 ms

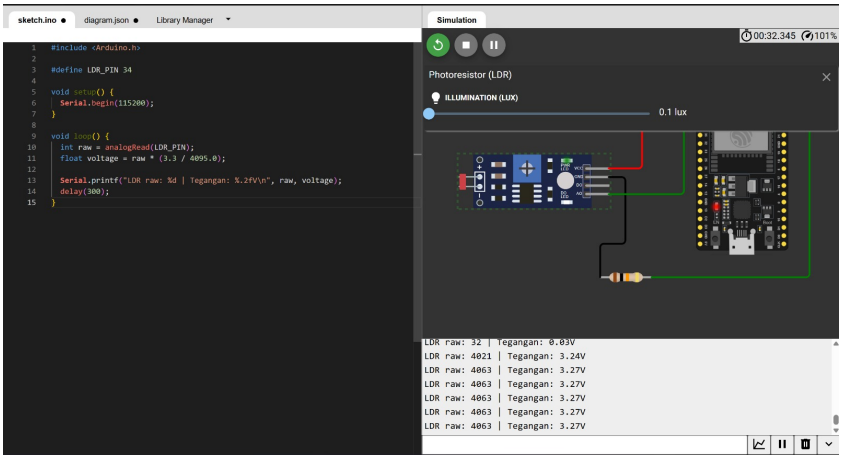
1.5 Percobaan 5: Pembacaan ADC dengan Sensor LDR

1. Membuat rangkaian pembagi tegangan antara sensor LDR dan resistor tetap 10kΩ, dengan titik tengah dihubungkan ke pin analog GPIO 34 ESP32.
2. Membaca nilai ADC 12-bit (rentang 0 sampai 4095) menggunakan fungsi `analogRead(34)` lalu mengonversinya ke estimasi tegangan dengan rumus $V = raw \times (3.3/4095.0)$.

3. Menguji pembacaan sensor pada simulator Wokwi di dua kondisi:
    - Kondisi gelap penuh (*Total Cover*), seperti pada Gambar 9.
    - Kondisi terang penuh (*Total Illumination*), seperti pada Gambar 10.



Gambar 9: [SIMULASI WOKWI] Pembacaan ADC Sensor LDR pada Kondisi Gelap Penuh



Gambar 10: [SIMULASI WOKWI] Pembacaan ADC Sensor LDR pada Kondisi Terang Penuh

2 Analisis Hasil Percobaan

2.1 Analisis Percobaan 1: STM32 Blackpill (Blink Arduino)

Program Blink berhasil dijalankan pada STM32 Blackpill menggunakan programmer ST-Link V2. Pin PC13 terhubung ke LED onboard dengan konfigurasi *active-low*, sehingga logika LOW menyalakan LED dan logika HIGH mematikannya. Framework Arduino (STM32duino) di PlatformIO mempermudah pengontrolan pin lewat fungsi `digitalWrite()`, tanpa harus mengatur register periferal RCC maupun konfigurasi clock ARM secara manual.

2.2 Analisis Percobaan 2: ESP32 (Blink ESP-IDF)

Percobaan Blink pada ESP32 dijalankan menggunakan framework native ESP-IDF. Berbeda dengan Arduino yang memakai alur `setup()` dan `loop()`, ESP-IDF berjalan langsung di atas FreeRTOS de-

ngan fungsi utama `app_main()`. Kontrol LED menggunakan driver `gpio_set_level()`, dan jeda waktu diatur melalui `vTaskDelay(pdMS_TO_TICKS(500))`. Penggunaan `vTaskDelay()` menempatkan task ke kondisi *blocked*, sehingga prosesor tidak tertahan dalam siklus sibuk (*busy-waiting*) dan bebas mengeksekusi proses lain.

2.3 Analisis Percobaan 3: Pull-up dan Pull-down Eksternal dan Internal

Percobaan ini menguji respon logika tombol dengan resistor pull-up dan pull-down:

- **Kendala Hardware di Lab:** Saat pengujian di lab, tombol pull-down (GPIO 33) terbaca berlogika HIGH terus-menerus. Hal ini terjadi karena tombol tactile 4 kaki terpasang berputar 90 derajat pada breadboard. Kaki internal saklar yang saling terhubung membuat pin GPIO 33 tersambung permanen ke jalur 3.3V tanpa melewati mekanisme tombol.
- **Hasil Perbaikan Rangkaian Eksternal:** Setelah orientasi kaki tombol diperbaiki di simulator Wokwi, pembacaan tombol bekerja sesuai teori. Tombol pull-up menghasilkan nilai HIGH saat dilepas dan LOW saat ditekan. Sebaliknya, tombol pull-down menghasilkan nilai LOW saat dilepas dan HIGH saat ditekan.
- **Hasil Pengujian Rangkaian Internal:** Penggunaan fitur pull-up (`INPUT_PULLUP`) dan pull-down internal (`INPUT_PULLDOWN`) pada ESP32 memberikan hasil logika yang sama persis dengan rangkaian eksternal, sehingga menghemat pemakaian resistor tambahan pada breadboard.

2.4 Analisis Percobaan 4: Karakteristik Debouncing

Bilah mekanik saklar menghasilkan getaran sesaat (*bouncing*) saat ditekan atau dilepas. Hasil pengujian menunjukkan:

- Variabel `counterNoDebounce` rawan bertambah lebih dari 1 angka dalam sekali penekanan karena mendeteksi osilasi sinyal naik-turun yang terjadi sangat cepat.
- Variabel `counterWithDebounce` berhasil menyaring derau tersebut, sehingga pencacah selalu bertambah tepat 1 kali pada setiap penekanan stabil.
- Mengenai variasi waktu: jeda 10 ms masih berisiko jika tombol fisik sudah aus, jeda 50 ms merupakan batas optimal yang stabil tanpa jeda yang mengganggu, sedangkan jeda 200 ms menimbulkan latensi respon yang terasa lambat saat tombol ditekan berulang secara cepat.

2.5 Analisis Percobaan 5: Pembacaan ADC Sensor LDR

LDR dipadukan dengan resistor 10kΩ untuk membentuk rangkaian pembagi tegangan:

- Saat kondisi gelap (*Total Cover*), resistansi LDR meningkat drastis. Tegangan yang masuk ke GPIO 34 mendekati 0V, sehingga nilai ADC terbaca rendah di kisaran 0 hingga 50 dengan estimasi tegangan sekitar 0.00 V.
- Saat kondisi terang (*Total Illumination*), resistansi LDR turun drastis. Tegangan yang masuk ke GPIO 34 mendekati 3.3V, sehingga nilai ADC terbaca mendekati batas maksimum 12-bit (sekitar 4095) dengan estimasi tegangan 3.30 V.

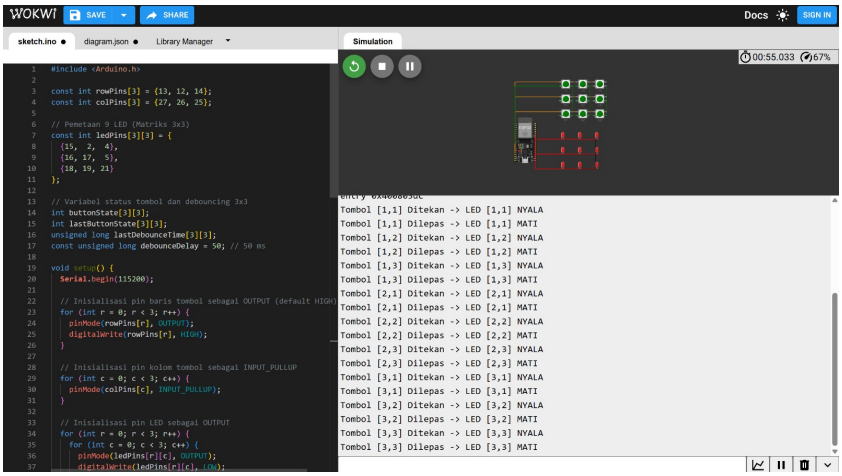
Hasil ini menunjukkan bahwa ADC membaca tegangan analog secara kontinu dalam 4096 tingkatan kuantisasi, berbeda dengan input digital yang hanya membaca dua kondisi biner.

3 Hasil Tugas Modul

3.1 Deskripsi dan Rangkaian Simulasi

Tugas modul mengimplementasikan matriks tombol 3x3 untuk mengendalikan 9 LED pada posisi yang bersesuaian, lengkap dengan algoritma debouncing mandiri pada tiap tombol.

- **Pin Baris Tombol (Output):** Row 1 = GPIO 13, Row 2 = GPIO 12, Row 3 = GPIO 14.
- **Pin Kolom Tombol (Input Pull-up):** Col 1 = GPIO 27, Col 2 = GPIO 26, Col 3 = GPIO 25.
- **Pin LED (Output):** GPIO 15, 2, 4, 16, 17, 5, 18, 19, dan 21.



Gambar 11: [SIMULASI WOKWI] Hasil Pengujian Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

3.2 Source Code Program

Listing 1: Source Code Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3 dengan Debouncing

```
#include <Arduino.h>

const int rowPins[3] = {13, 12, 14};
const int colPins[3] = {27, 26, 25};

// Pemetaan 9 LED (Matriks 3x3)
const int ledPins[3][3] = {
  {15,  2,  4},
  {16, 17,  5},
  {18, 19, 21}
};

int buttonState[3][3];
int lastButtonState[3][3];
unsigned long lastDebounceTime[3][3];
const unsigned long debounceDelay = 50; // Jeda stabil 50 ms

void setup() {
  Serial.begin(115200);

  // Inisialisasi pin baris sebagai OUTPUT (default HIGH)
  for (int r = 0; r < 3; r++) {
```

```
    pinMode(rowPins[r], OUTPUT);
    digitalWrite(rowPins[r], HIGH);
  }

  // Inisialisasi pin kolom sebagai INPUT_PULLUP
  for (int c = 0; c < 3; c++) {
    pinMode(colPins[c], INPUT_PULLUP);
  }

  // Inisialisasi pin LED sebagai OUTPUT
  for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 3; c++) {
      pinMode(ledPins[r][c], OUTPUT);
      digitalWrite(ledPins[r][c], LOW);
      buttonState[r][c] = HIGH;
      lastButtonState[r][c] = HIGH;
      lastDebounceTime[r][c] = 0;
    }
  }
}

void loop() {
  // Scanning baris satu per satu
  for (int r = 0; r < 3; r++) {
    digitalWrite(rowPins[r], LOW); // Tarik baris aktif ke LOW

    for (int c = 0; c < 3; c++) {
      int reading = digitalRead(colPins[c]);

      if (reading != lastButtonState[r][c]) {
        lastDebounceTime[r][c] = millis();
      }

      if ((millis() - lastDebounceTime[r][c]) > debounceDelay) {
        if (reading != buttonState[r][c]) {
          buttonState[r][c] = reading;

          if (buttonState[r][c] == LOW) {
            digitalWrite(ledPins[r][c], HIGH);
            Serial.printf("Tombol [%d,%d] Ditekan -> LED [%d,%d] NYALA\n", r+1, c+1,
    r+1, c+1);
          } else {
            digitalWrite(ledPins[r][c], LOW);
            Serial.printf("Tombol [%d,%d] Dilepas -> LED [%d,%d] MATI\n", r+1, c+1, r
    +1, c+1);
          }
        }
      }
      lastButtonState[r][c] = reading;
    }

    digitalWrite(rowPins[r], HIGH); // Kembalikan baris ke HIGH
  }
}
```

3.3 Jawaban Pertanyaan Analisis Tugas Modul

1. **Mengapa debouncing tetap diperlukan pada matrix pushbutton ini walaupun hanya satu tombol yang ditekan?** Debouncing tetap diperlukan karena getaran mekanis (*bouncing*) terjadi di dalam saklar tombol itu sendiri saat pelat logam bersentuhan. Scanning matriks berjalan sangat cepat dalam orde mikrodetik, sehingga tanpa filter waktu, mikrokontroler akan menangkap getaran tersebut sebagai rentetan penekanan cepat yang membuat LED berkedip dan status pemicuan tidak stabil.

2. **Konfigurasi pull-up/pull-down pada kolom, kondisi saat tidak ditekan, dan bahaya floating pin:** Rancangan menggunakan konfigurasi pull-up internal (`INPUT_PULLUP`) pada pin kolom. Saat tombol tidak ditekan, pin bernilai **HIGH**. Jika pin dibiarkan mengambang (*floating*), impedansi gerbang input CMOS yang sangat tinggi akan menangkap interferensi listrik statis sekitar, sehingga logika pin berfluktuasi liar dan memicu pembacaan penekanan palsu (*ghost trigger*).

3. **Perbedaan membaca GPIO sebagai digital input vs analog input (ADC):** Digital input hanya membaca dua kondisi diskret (0 atau 1) berdasarkan ambang batas tegangan logika CMOS (V_{IL} dan V_{IH}). Sedangkan ADC (seperti ADC 12-bit ESP32) mengonversi rentang tegangan 0-3.3V menjadi 4096 tingkatan angka diskret (0-4095). Hal ini memungkinkan pembacaan nilai fisis sensor secara bertahap dan kontinu.

4 Kesimpulan

1. PlatformIO pada VS Code mempermudah manajemen proyek dan kompilasi firmware untuk dua target arsitektur berbeda (STM32 dan ESP32).
2. STM32 Blackpill memerlukan programmer eksternal ST-Link V2 melalui antarmuka SWD, sedangkan ESP32 dapat langsung diprogram via USB berkat chip bridge USB-to-UART dan rangkaian auto-reset onboard.
3. Framework Arduino mempercepat pembuatan purwarupa lewat abstraksi fungsi tinggi, sedangkan ESP-IDF memberikan efisiensi kontrol hardware tingkat rendah di atas FreeRTOS.
4. Resistor pull-up dan pull-down penting untuk mencegah pin input berada dalam kondisi mengambang (*floating*). Orientasi pemasangan tombol 4 kaki pada breadboard harus tepat agar tidak terjadi kontak permanen ke rel tegangan.
5. Filter debouncing berbasis software berhasil meredam derau bouncing tombol mekanik, dengan jeda waktu stabil 50 ms sebagai nilai yang optimal.
6. Peripheral ADC 12-bit pada ESP32 mampu membaca variasi tegangan analog dari sensor LDR dan mengonversinya menjadi data digital dalam rentang 0 hingga 4095.

5 Lampiran

5.1 Dokumentasi saat Praktikum



Gambar 12: Dokumentasi Pelaksanaan Praktikum Modul 1 di Laboratorium A108

Daftar Pustaka

[1] STMicroelectronics, *STM32F401xB/C Advanced ARM-based 32-bit MCUs Reference Manual (RM0368)*, STMicroelectronics, 2021. **url**: `https://www.st.com/`

[2] STMicroelectronics, *UM1724 User Manual: ST-LINK/V2 in-circuit debugger/programmer for STM8 and STM32*, STMicroelectronics, 2020. **url**: `https://www.st.com/`

[3] Espressif Systems, *ESP-IDF Programming Guide*, Espressif Systems, 2023. **url**: `https://docs.espressif.com/projects/esp-idf/`

[4] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems, 2024. **url**: `https://www.espressif.com/`

[5] STM32duino Community. “Arduino Core for STM32 Documentation. ”**url**: `https://github.com/stm32duino/Arduino_Core_STM32`

[6] PlatformIO Labs. “PlatformIO Documentation. ”**url**: `https://docs.platformio.org/`

[7] J. Ganssle, “A Guide to Debouncing, or, How to Debounce a Contact in Two Easy Pages and 19 Tricky Ones,” *The Ganssle Group*, 2014. **url**: `http://www.ganssle.com/debouncing.htm`